

UNIVERSITÀ DEGLI STUDI DI PADOVA

Corso di Laurea in Informatica
Insegnamento di Ingegneria del Software

Anno Accademico 2025/2026

Specifica Tecnica

Architettura e Design di Sistema

Gruppo: Synergo Unit

Email: synergounit20@gmail.com

Versione: **v0.3.1**
Data: **2026-08-06**

Registro delle Modifiche

Versione	Data	Autore	Verificatore	Descrizione
0.3.0	2026-08-08	Sofia De Blasi	Francesco Marcon	Stesura sezioni Interfaccia API, Design Pattern Utilizzati, Flusso dei Dati e Tracciamento dei Requisiti con diagrammi.
0.2.0	2026-08-08	Sofia De Blasi	Francesco Marcon	Stesura sezioni Tecnologie e Architettura con diagrammi.
0.1.0	2026-08-06	Sofia De Blasi	Francesco Marcon	Stesura Introduzione.
0.0.0	2026-08-06	Sofia De Blasi	Francesco Marcon	Prima stesura della struttura del documento.

Indice

1	Introduzione	6
1.1	Scopo del documento	6
1.2	Scopo del prodotto	6
1.3	Glossario	6
1.4	Riferimenti	7
1.4.1	Normativi	7
1.4.2	Informativi	7
2	Tecnologie Utilizzate	8
2.1	Linguaggi, runtime e framework	8
2.2	Librerie Frontend	8
2.3	Librerie Backend	9
2.4	Persistenza dei dati	9
2.5	Testing	9
2.6	Infrastruttura e strumenti di sviluppo	9
3	Architettura del Sistema	10
3.1	Architettura Generale	10
3.2	Architettura di Deployment	10
3.3	Architettura del Frontend	11
3.3.1	Gestione dello Stato e dell'Editor	12
3.3.2	Gestione Interazioni AI (Widget/Pannello AI)	12
3.4	Architettura del Backend	13
3.4.1	Livello di Presentazione	14
3.4.2	Livello di Business Logic (Core LLM)	14
3.4.3	Componente di Esportazione (Export)	15
4	Interfaccia API e Comunicazione	17
4.1	Struttura dei Payload AI	17
4.2	Gestione degli Errori e Fallback	17
5	Design Pattern Utilizzati	19
5.1	Pattern Creazionali	19
5.1.1	Simple Factory	19
5.1.2	Builder	20
5.2	Pattern Strutturali	20
5.2.1	Decorator	20

5.2.2	Adapter	21
5.2.3	Proxy (remoto)	21
5.2.4	Template Method	22
5.3	Pattern Comportamentali	22
5.3.1	Strategy	22
5.3.2	Command	23
5.3.3	Observer	23
5.3.4	Dependency Injection	24
6	Flusso dei Dati (Data Flow)	25
6.1	Elaborazione di un'operazione AI	25
6.2	Accettazione della proposta ed elenco delle operazioni	26
6.3	Editing del testo e Undo/Redo	28
6.4	Persistenza locale della nota	30
6.5	Esportazione	31
7	Tracciamento e Stato dei Requisiti	33
7.1	Tabella Tracciamento Componenti-Requisiti	33
7.2	Stato di Soddisfacimento dei Requisiti	34

Elenco delle figure

1	Diagramma dei componenti del sistema.	10
2	Diagramma UML di Deployment.	11
3	Diagramma dei package del Frontend.	11
4	Diagramma delle classi del Frontend (Model, View, Controller, Proxy).	13
5	Diagramma dei package del Backend.	14
6	Diagramma delle classi del livello di Presentazione e DI del Backend.	14
7	Diagramma delle classi del dominio AI: operazioni, costruzione dei prompt e integrazione con il servizio LLM.	15
8	Diagramma delle classi del package Export (Template Method).	16
9	Richiesta operazione AI (tratto Frontend).	25
10	Richiesta AI (tratto REST→LLM): attraversa in un unico flusso i pattern Strategy, Builder, Simple Factory, Decorator e Adapter.	26
11	Ricezione proposta AI (tratto Frontend).	26
12	Accetta proposta AI (riuso del pattern Command).	27
13	Elenco operazioni disponibili (Simple Factory self-registration).	27
14	Formattazione di un testo e relativo Undo.	28
15	Redo (continuazione di “Formattazione con Undo”).	29
16	Inserimento di una tabella, con validazione delle dimensioni.	29
17	Salvataggio di una nota in locale.	30
18	Salvataggio con errore di I/O.	31
19	Importazione/Apertura di una nota esistente.	31
20	Esportazione di una nota in PDF (Template Method).	32

Elenco delle tabelle

1	Tracciamento Componenti → Requisiti	33
2	Stato dei Requisiti	34

1 Introduzione

1.1 Scopo del documento

Il presente documento ha lo scopo di definire l'architettura di base e di dettaglio del sistema **Second Brain**. Vengono illustrate le scelte tecnologiche effettuate dal gruppo Synergo Unit durante la fase di Product Baseline, i pattern architetturali e di design adottati, la decomposizione in componenti del sistema e il tracciamento bidirezionale tra queste componenti e i requisiti individuati nella fase di Analisi.

A differenza della Technology Baseline, in cui l'obiettivo era esclusivamente validare la fattibilità tecnica delle scelte effettuate tramite un Proof of Concept, questo documento consolida un'architettura definitiva, coerente con il PoC ma estesa e rifinita per coprire la totalità dei requisiti obbligatori individuati in [Analisi dei Requisiti](#) (Sezione 4 - Requisiti).

1.2 Scopo del prodotto

Second Brain è un'applicazione web per la redazione di testi in formato Markdown, pensata per superare i limiti dei classici chatbot testuali fornendo un editor integrato in cui un Large Language Model (LLM) può attivamente elaborare, criticare e generare parti di testo su richiesta dell'utente.

Le funzionalità principali riguardano:

- la scrittura e formattazione di testo Markdown (grassetto, corsivo, sottolineato, barrato, citazioni, intestazioni, link, elenchi, tabelle) con anteprima in tempo reale e cronologia di annullamento/ripristino;
- il supporto AI alla scrittura: riassunto, traduzione (Inglese, Francese, Spagnolo) e riscrittura del testo selezionato;
- l'analisi critica del testo secondo il metodo dei “Sei Cappelli per Pensare” di Edward de Bono (dati e fatti, emozioni, rischi, opportunità, creatività, logica/controllo);
- il *Distant Writing*: la generazione di interi blocchi di testo a partire da un prompt fornito dall'utente in una finestra dedicata;
- la gestione dei dati: salvataggio, apertura ed esportazione (PDF, HTML, JSON) delle note come requisito obbligatorio basato sul file system locale, con gestione di note remote, autenticazione utente e persistenza server-side come estensione opzionale.

L'utente di riferimento è un individuo senza competenze tecniche pregresse (tipicamente uno studente) che utilizza l'applicazione per prendere appunti e desidera supporto nella loro elaborazione tramite LLM; l'interfaccia deve pertanto risultare semplice e intuitiva. L'unico attore secondario del sistema è il servizio LLM esterno, richiamato tramite API compatibile con lo standard OpenAI.

1.3 Glossario

Al fine di evitare ambiguità, i termini tecnici o con significato specifico sono seguiti dal pedice e definiti nel documento [Glossario](#).

1.4 Riferimenti

1.4.1 Normativi

- [Capitolato C6 – Second Brain \(Zucchetti S.p.A.\)](#)
Ultimo accesso: 2026-08-06;
- [Norme di Progetto, v2.12.0;](#)

1.4.2 Informativi

- [Analisi dei Requisiti](#), v1.8.0, in particolare la Sezione 3 (Casi d'Uso) e la Sezione 4 (Requisiti);
- [Analisi dello Stato dell'Arte](#), v1.0.0, in particolare la Sezione 2 (Tecnologie Frontend), la Sezione 3 (Tecnologie Backend) e la Sezione 4 (Struttura Tecnica Preliminare del Proof of Concept);
- [Piano di Qualifica](#), v1.7.0;
- [Vue.js Guide – Reactivity Fundamentals](#)
Ultimo accesso: 2026-08-06;
- [CodeMirror – System Guide](#)
Ultimo accesso: 2026-08-06;
- [FastAPI – Dependencies](#)
Ultimo accesso: 2026-08-08;
- [OpenAI API Reference](#), pagina *Create chat completion* – standard di riferimento per l'interazione con i modelli LLM (R2-V-O)
Ultimo accesso: 2026-08-08;
- [Docker Docs – Docker Compose](#)
Ultimo accesso: 2026-08-03;

2 Tecnologie Utilizzate

Le tecnologie adottate sono state selezionate secondo i criteri motivati in dettaglio in [Analisi dello Stato dell'Arte](#) (Sezioni 2–3), a cui si rimanda per l'analisi comparativa delle alternative scartate. Le versioni riportate corrispondono ai valori definiti nei file di lock del progetto (`package-lock.json`, `requirements.txt`) alla data di redazione di questo documento.

2.1 Linguaggi, runtime e framework

Tecnologia	Versione	Descrizione
TypeScript	6.0.x	Linguaggio principale del Frontend; la tipizzazione statica riduce gli errori di integrazione con le interfacce dei payload AI.
Vue.js	3.5.x	Framework a componenti del Frontend, scelto per la progressività e la bassa curva di apprendimento (Sezione 2 di Analisi dello Stato dell'Arte).
Python	3.11	Linguaggio del Backend, scelto per la maturità del suo ecosistema nell'integrazione con strumenti di intelligenza artificiale e NLP per per la pregressa conoscenza da parte di alcuni membri del gruppo.
FastAPI	0.136.3	Framework Backend per API REST asincrone; espone i router <code>AIRouter</code> ed <code>ExportRouter</code> (Sezione 4).

2.2 Librerie Frontend

Libreria	Versione	Descrizione
Pinia	3.0.4	Libreria di gestione dello stato globale dell'applicazione Vue.js.
vue-router	5.1.0	Libreria di routing lato client tra le viste dell'applicazione.
CodeMirror	6.0.2	Componente editor di testo integrato nel Frontend (Sezione 2 di Analisi dello Stato dell'Arte).
vue-codemirror6	1.5.2	Binding Vue.js per l'integrazione di CodeMirror come componente reattivo.
marked	18.0.4	Conversione del contenuto Markdown della nota in HTML per l'anteprima.
DOMPurify	3.4.8	Sanitizzazione dell'HTML generato da <code>marked</code> prima del rendering, a protezione da XSS.

2.3 Librerie Backend

Libreria	Versione	Descrizione
Pydantic	2.13.4	Validazione dei dati e definizione tipizzata dei DTO delle operazioni AI (<code>AIOperationRequest</code> , <code>AIOperationResponse</code> , <code>ExportRequest</code> ; Sezione 4), integrata nativamente in FastAPI.
openai	2.41.0	Client ufficiale per l'API compatibile con lo standard OpenAI (R2-V-O, R3-V-O); configurato con <code>base_url</code> verso l'endpoint fornito dall'azienda proponente.
httpx	0.28.1	Libreria HTTP asincrona, dipendenza di <code>openai</code> per le chiamate al servizio LLM esterno.
Uvicorn	0.49.0	Server ASGI su cui viene eseguito il Backend FastAPI.
python-dotenv	1.2.2	Caricamento delle variabili d'ambiente (<code>ZUCCHETTI_LLM_BASE_URL</code> , <code>ZUCCHETTI_LLM_API_KEY</code>) da file <code>.env</code> , per non includere credenziali nel codice sorgente.

2.4 Persistenza dei dati

Il gruppo ha scelto di non implementare alcun sistema di persistenza server-side in questa fase: la gestione dei dati si basa esclusivamente sul file system locale del browser (salvataggio, apertura ed esportazione della nota, R75–R77), che copre i requisiti obbligatori del capitolato.

La persistenza remota delle note e la gestione di utenti autenticati sono funzionalità opzionali (R4-V-Op, R78–R86) che il gruppo ha deciso di non realizzare, per concentrare le risorse disponibili sul consolidamento dei requisiti obbligatori; non è pertanto presente alcun database nello stack tecnologico né nell'architettura di deployment (Sezione 3.2). Lo stato di questi requisiti opzionali è riportato in dettaglio in Sezione 7.

2.5 Testing

Framework	Descrizione
Vitest	Framework di test per il Frontend TypeScript/Vue.js.
pytest	Framework di test per il Backend Python.

2.6 Infrastruttura e strumenti di sviluppo

Strumento	Descrizione
<u>Docker Compose</u>	Orchestrazione dei container frontend e backend (Sezione 3.2). Il server di sviluppo Vite del container frontend inoltra le richieste verso <code>/api</code> al container backend (<code>http://backend:8000</code>); non è presente un reverse proxy dedicato (es. Nginx) in questa fase.
Git e GitHub	Versionamento del codice e gestione collaborativa del repository, con tracciamento delle attività tramite issue (requisito R5-Q-O).
GitHub Actions	Automazione di build, esecuzione dei test e controlli di qualità a ogni push/pull request (requisiti R1-Q-O, R6-Q-O).
StarUML	Modellazione UML e produzione dei diagrammi architetturali.

3 Architettura del Sistema

3.1 Architettura Generale

Il sistema *Second Brain* è basato su un'architettura di tipo Client-Server per garantire la separazione delle responsabilità (*Separation of Concerns*) tra la gestione dell'interfaccia utente (editor Markdown) e l'orchestrazione delle chiamate agli LLM.

Il sistema è composto da due soli componenti software: **Web Application** (Frontend) e **Backend API**, che comunicano tramite un'unica interfaccia REST (**IBackendREST**). Il Frontend non comunica mai direttamente con il servizio LLM: infatti, ogni richiesta di elaborazione AI transita dal Backend, che funge da unico punto di orchestrazione verso il provider esterno. Questa scelta, oltre a rispettare il vincolo R2-V-O sullo standard delle chiamate API, evita di esporre nel browser eventuali credenziali di accesso al servizio LLM.

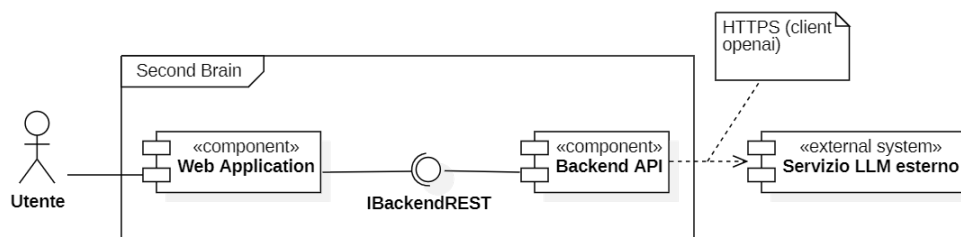


Figura 1: Diagramma dei componenti del sistema.

3.2 Architettura di Deployment

Frontend e Backend sono eseguiti in due container Docker distinti, orchestrati tramite Docker Compose:

- **frontend**: applicazione Vue.js, servita in sviluppo dal dev server Vite sulla porta 5173;
- **backend**: servizio FastAPI eseguito tramite Uvicorn sulla porta 8000, espone le API REST consumate dal frontend e contatta il servizio LLM esterno tramite HTTPS, con la relativa API key iniettata come variabile d'ambiente.

Questa scelta di deployment corrisponde a un'**architettura a due moduli deployabili indipendentemente (*two-monolith*)**, ciascuno containerizzato separatamente ma non ulteriormente suddiviso al proprio interno.

È stata preferita tale architettura piuttosto che:

- un **monolite unico indifferenziato**, perchè avrebbe accoppiato il deploy del Frontend (statico) a quello del Backend (con dipendenze verso servizi LLM esterni), impedendo aggiornamenti o riavvii indipendenti dei due componenti;
- una **scomposizione a microservizi** (es. un servizio per ciascuna operazione AI) perchè non sarebbe giustificata dalla scala del sistema, che espone un dominio applicativo singolo e un volume di richieste limitato, inoltre, introdurrebbe complessità di orchestrazione (service discovery, API gateway) senza benefici concreti, tanto più in assenza di persistenza (Sezione 2.4).

La granularità scelta consente comunque deploy e scalabilità indipendenti dei due componenti, preservando la possibilità di un'evoluzione più granulare qualora emergessero nuovi requisiti.

Il server di sviluppo Vite del container **frontend** inoltra le richieste verso `/api` al container **backend** attraverso la rete Docker interna; non è presente alcun container di persistenza, coerentemente con la scelta descritta in Sezione 2.4 di non implementare la persistenza remota (R4-V-Op, R78–R86).

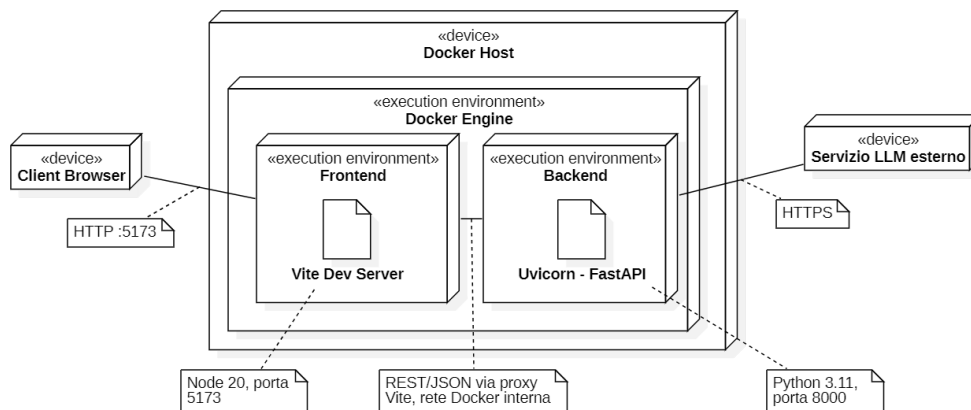


Figura 2: Diagramma UML di Deployment.

3.3 Architettura del Frontend

Il Frontend gestisce l'interazione dell'utente con l'editor Markdown e la visualizzazione dello stato delle operazioni AI. Il pattern architetturale adottato è **Model-View-Controller** nella versione *Pull Model*.

Questa separazione esplicita, indipendente dal framework di rendering, mantiene la logica di dominio (Model e Controller) testabile in isolamento: i componenti Vue.js costituiscono l'implementazione concreta della View, limitandosi a riflettere nel template lo stato esposto dal relativo oggetto View e a inoltrare gli eventi dell'utente, senza contenere essi stessi logica applicativa.

Vue.js è stato scelto consapevole che, a differenza di framework più strutturati, non fornisce nativamente un sistema di Dependency Injection né impone pattern architetturali quindi questi sono stati progettati e introdotti autonomamente dal gruppo (Sezione 5).

L'orchestrazione tra View e Model è affidata a due Controller distinti, coerenti con la separazione tra editing e interazioni AI: **EditorController** media tra **EditorView** e **NoteModel** (creazione ed esecuzione dei comandi di editing), mentre **AIController** media tra **AIPanelView** e **AIRquestModel** (avvio delle richieste AI e gestione delle proposte).

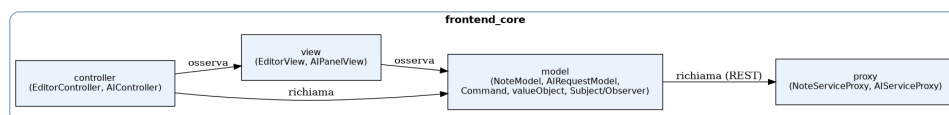


Figura 3: Diagramma dei package del Frontend.

3.3.1 Gestione dello Stato e dell'Editor

Il `NoteModel` mantiene lo stato osservabile della nota corrente (contenuto, identificativo, flag di modifica non salvata) e delega la manipolazione testuale a un `MarkdownContentEditor`. Ogni modifica (formattazione, tabelle, elenchi, link, inserimento di testo) è incapsulata in un comando (Sezione 5) eseguito tramite `NoteModel.executeCommand()`, che aggiorna una `CommandHistory` a due pile per supportare Undo/Redo (R43-F-O, R44-F-O) in modo uniforme per tutte le operazioni di editing.

Il salvataggio e l'apertura della nota (R75-F-O, R76-F-O) sono delegati tramite l'interfaccia `NoteService` a un'implementazione basata sul File System locale, così da non vincolare il Model a un meccanismo di persistenza specifico; la scelta tra creazione di un nuovo file (`showSaveFilePicker`) e sovrascrittura di un file già aperto è gestita internamente al `NoteModel` in base alla presenza di un identificativo di nota.

3.3.2 Gestione Interazioni AI (Widget/Pannello AI)

L'`AIRequestModel` mantiene lo stato della richiesta AI corrente come un tipo dato algebrico (union discriminata `TypeScript`) con quattro varianti: `IdleState`, `ProcessingState`, `ProposalReadyState` (con la `Proposal` generata) ed `ErrorState`. Abbiamo preferito un'unione discriminata al pattern State GoF, che avrebbe introdotto una gerarchia di classi senza incapsulare alcun comportamento reale: le varianti (`IdleState`, `ProcessingState`, `ProposalReadyState`, `ErrorState`) sono semplici contenitori di dati privi di metodi polimorfici, mentre la logica di transizione resta centralizzata in `AIRequestModel`.

Il ciclo di vita di una richiesta AI attraversa tre fasi distinte, ciascuna illustrata da un diagramma di sequenza dedicato in Sezione 6:

1. l'utente avvia la richiesta e riceve un riscontro visivo immediato (stato `Processing`, R72-F-O) con l'opzione di interruzione manuale (R73-F-O);
2. il Backend elabora la richiesta interagendo con l'LLM;
3. una volta risolta la Promise di `AIServiceProxy.requestOperation()`, aggiorna lo stato a `ProposalReadyState` e la View mostra la proposta con le azioni disponibili.

L'utente può quindi accettare, rifiutare o rigenerare la proposta (R69–R71): l'accettazione riusa lo stesso meccanismo di comando dell'editing manuale, inserendo il testo proposto tramite un comando eseguito su `NoteModel` (pattern Command).

Le richieste verso il Backend sono mediate dall'interfaccia `AIService`, la cui implementazione concreta (Proxy) incapsula i dettagli della chiamata REST.

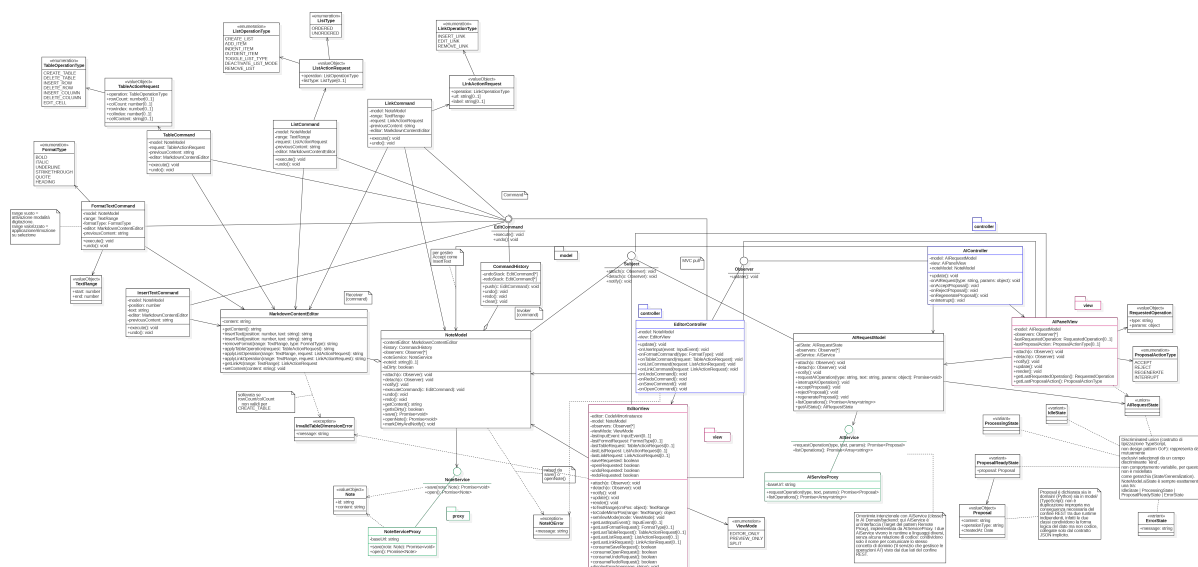


Figura 4: Diagramma delle classi del Frontend (Model, View, Controller, Proxy).

3.4 Architettura del Backend

Il Backend funge da orchestratore centrale e da strato di astrazione verso i provider LLM esterni. È organizzato secondo un'architettura a livelli, articolata in due livelli principali:

1. **Livello di Presentazione** (package `backend.API_business_layer`): router REST, DTO, Dependency Injection;
2. **Livello di Business Logic** (package `AI_Domain`): logica di costruzione dei prompt e di integrazione con il servizio LLM esterno, indipendente dal framework web.

A questi si affianca il package **Export**, che non fa parte della catena di elaborazione delle richieste AI, bensì implementa in modo isolato e autosufficiente la conversione della nota nei formati richiesti, riusando lo stesso principio di separazione delle responsabilità senza costituire un ulteriore livello architetturale.

La **scomposizione a livelli** riflette la scala contenuta del sistema garantendo una separazione sufficiente a mantenere il dominio AI indipendente sia dal framework HTTP che dal provider LLM concreto.

La scelta tra un'architettura Esagonale e una a Layer è stata oggetto di un confronto esplicito con il referente aziendale (Zucchetti, verbale del 30/07/2026, decisione VE-8.1): sono stati messi a confronto un'architettura Esagonale, più pulita e flessibile rispetto a possibili evoluzioni future ma a fronte di una maggiore complessità realizzativa, e un'architettura a Layer, più naturale per applicazioni di questo tipo e meno onerosa da realizzare quando non sono previste evoluzioni architetturali significative. Non essendo previste, per *Second Brain*, estensioni quali la persistenza remota o l'integrazione con provider LLM multipli all'interno della stessa richiesta (R4-V-Op, R6-V-O), è stata adottata l'architettura a Layer, riservando comunque, tramite la separazione tra Presentazione e Business Logic già descritta, un margine di evoluzione verso un'architettura più disaccoppiata qualora emergessero nuovi requisiti.

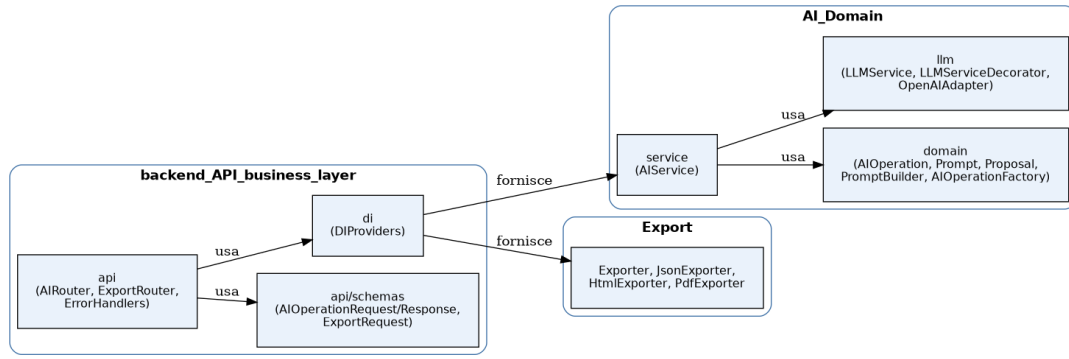


Figura 5: Diagramma dei package del Backend.

3.4.1 Livello di Presentazione

I router FastAPI (`AIRouter`, `ExportRouter`) ricevono le richieste dal client, le validano tramite i DTO di `api/schemas` (`AIOperationRequest`, `ExportRequest`) e delegano l'elaborazione ai servizi applicativi, ottenuti tramite Dependency Injection (`DIProviders`). Un gestore di errori centralizzato (`ErrorHandlers`) traduce le eccezioni di dominio (`AIDomainError`, `ExportError`) in risposte HTTP coerenti (Sezione 4.2). I router non contengono logica di dominio: traducono richieste HTTP in chiamate al livello di Business Logic e le risposte in schemi Pydantic.

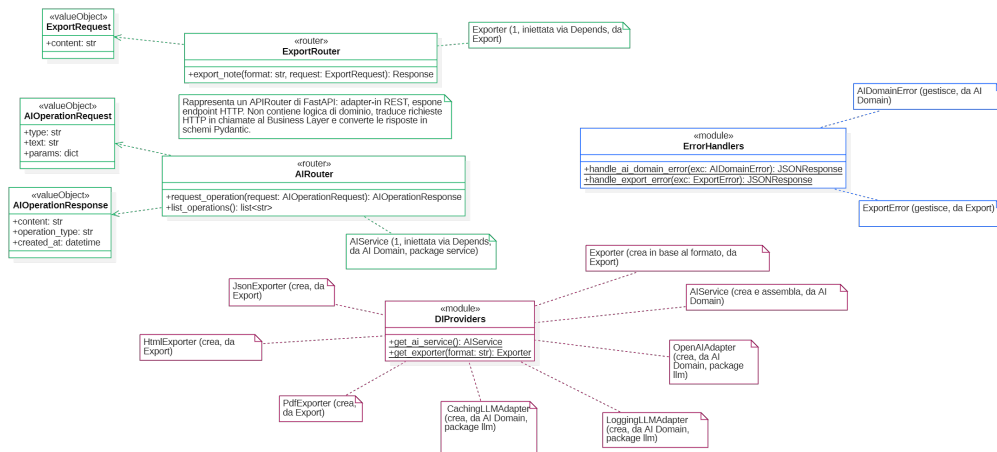


Figura 6: Diagramma delle classi del livello di Presentazione e DI del Backend.

3.4.2 Livello di Business Logic (Core LLM)

L'`AIService` è il motore centrale del dominio AI: riceve il tipo di operazione richiesta (riassunto, traduzione, riscrittura, Distant Writing, o una delle sei prospettive dei “Cappelli di De Bono”), la risolve tramite un'`AIOperationFactory` auto-registrante, e ne invoca `build_prompt()` in modo polimorfo. Ogni operazione costruisce il proprio `Prompt` tramite un `PromptBuilder` fluente, isolando la logica specifica di ciascuna trasformazione testuale (Sezione 5, pattern Strategy, Builder e Simple Factory).

Il pacchetto `AI_Domain/llm`, incluso nello stesso diagramma, maschera le specificità del provider LLM concreto dietro l'interfaccia `LLMSERVICE`: una catena di decorator (`CachingLLMAdapter`, `LoggingLLMAdapter`) aggiunge funzionalità trasversali di caching e logging senza modificare l'im-

plementazione di base; l'ultimo anello della catena è l'**OpenAIAdapter**, verso l'API compatibile con lo standard OpenAI (R2-V-O, R3-V-O).

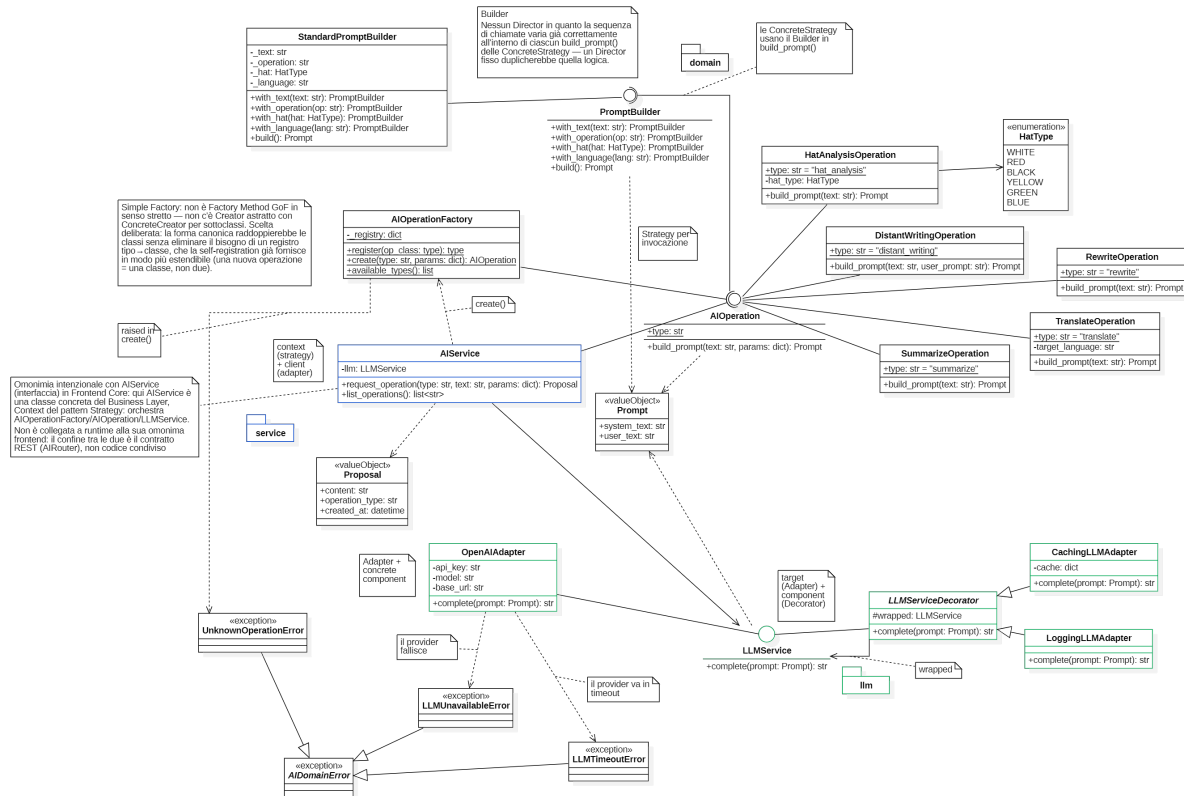


Figura 7: Diagramma delle classi del dominio AI: operazioni, costruzione dei prompt e integrazione con il servizio LLM.

3.4.3 Componente di Esportazione (Export)

La conversione della nota nei formati di esportazione richiesti (PDF, HTML, JSON)(R77-F-O) è isolata nel package **Export**, strutturato attorno al pattern Template Method (Sezione 5): la classe astratta **Exporter** fissa i passi comuni dell'esportazione (`export()`, `prepare_content()`) mentre delega alle sottoclassi (**PdfExporter**, **HtmlExporter**, **JsonExporter**) solo la conversione nel formato finale (`convert_format()`). `DIProviders.get_exporter(format)` risolve l'esportatore concreto in base al formato richiesto da **ExportRouter**, senza che **Export** debba conoscere né il livello di Presentazione né quello di Business Logic.

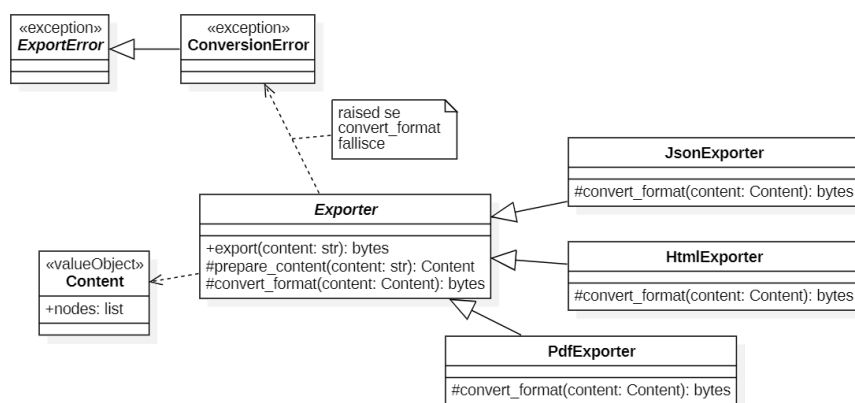


Figura 8: Diagramma delle classi del package Export (Template Method).

4 Interfaccia API e Comunicazione

Al fine di garantire il disaccoppiamento tra l'editor Markdown (Frontend) e l'elaboratore di prompt (Backend), la comunicazione avviene tramite API **REST** su JSON, esposta dal Backend FastAPI e consumata dal Frontend tramite il pattern Proxy (Sezione 5).

4.1 Struttura dei Payload AI

La comunicazione relativa alle operazioni di Intelligenza Artificiale (riassunto, traduzione, riscrittura, Sei Cappelli, Distant Writing) è normalizzata su un'unica struttura di richiesta e di risposta, validata da FastAPI tramite modelli tipizzati (`api/schemas`), indipendentemente dall'operazione richiesta.

DTO	Campo	Descrizione
AIOperationRequest	type	Tipo di operazione richiesta (es. <code>summarize</code> , <code>translate</code> , <code>rewrite</code> , <code>distant_writing</code> , <code>hat_white</code> , ...), risolto dinamicamente da <code>AIOperationFactory</code> .
	text	Testo selezionato dall'utente su cui operare (contesto).
	params	Parametri aggiuntivi specifici dell'operazione (es. lingua di destinazione per la traduzione, prompt utente per il Distant Writing).
AIOperationResponse	content	Testo generato dall'LLM, restituito come proposta.
	operation_type	Tipo di operazione a cui la risposta si riferisce, per consentire al Frontend di associare correttamente la proposta alla richiesta.
	created_at	Timestamp di generazione, utile per l'ordinamento e l'eventuale invalidazione della proposta.
ExportRequest	content	Contenuto Markdown della nota da convertire nel formato di esportazione richiesto (PDF, HTML o JSON).

Questa struttura unificata consente di aggiungere nuove operazioni AI (rispettando l'Open Closed Principle, Sezione 5) senza introdurre nuovi endpoint o nuovi contratti di comunicazione in quanto è sufficiente registrare una nuova strategia lato Backend.

4.2 Gestione degli Errori e Fallback

Le anomalie sono modellate come una gerarchia di eccezioni di dominio (`AIDomainError`), distinta dalle eccezioni tecniche di rete o di parsing:

- **LLMUnavailableError**: il servizio LLM esterno non è raggiungibile (assenza di connettività o servizio non configurato);
- **LLMTimeoutError**: la richiesta al modello LLM eccede il timeout configurato;
- **UnknownOperationError**: il tipo di operazione richiesto non è registrato in **AIOperationFactory**.

Un gestore di eccezioni centralizzato (**ErrorHandlers**) intercetta queste eccezioni all'interno dei router e le traduce in risposte HTTP normalizzate (codice di stato e messaggio), che il Frontend interpreta per notificare l'utente in caso di errore durante una qualsiasi delle operazioni AI (R49-F-O, R51-F-O, R54-F-O, R56-F-O, R58-F-O, R60-F-O, R62-F-O, R64-F-O, R66-F-O, R68-F-O), mantenendo il documento invariato e permettendo il rifiuto o la rigenerazione della proposta (R70-F-O, R71-F-O).

Coerentemente con il vincolo R6-V-O, ogni richiesta di elaborazione AI è indirizzata a un singolo modello LLM per l'intera durata della richiesta: il Backend non concatena chiamate a modelli differenti all'interno della stessa operazione.

5 Design Pattern Utilizzati

Di seguito sono illustrati i design pattern adottati per risolvere problematiche ricorrenti e garantire la scalabilità del codice di *Second Brain*. Ogni pattern è stato introdotto solo laddove i diagrammi architetturali ne mostrano un utilizzo concreto.

5.1 Pattern Creazionali

5.1.1 Simple Factory

Problema risolto	AIService deve creare l'istanza dell'operazione AI corretta a partire da una stringa (il tipo di operazione richiesto dal Frontend) senza contenere un'istruzione condizionale per ogni tipo esistente, e nuove operazioni devono potersi aggiungere rispettando l'Open Closed Principle.
Soluzione adottata	AIOperationFactory mantiene un registro (<code>_registry</code>) e offre <code>register(op_class)</code> per l'auto-registrazione delle classi operazione e <code>create(type, params)</code> per istanziarle; solleva <code>UnknownOperationError</code> se il tipo non è registrato. Non è un Factory Method GoF in senso stretto: infatti, non esiste un Creator astratto con ConcreteCreator per sottoclasse e tale scelta dovuta dal fatto che la forma canonica del Factory Method raddoppierebbe le classi (una fabbrica per ogni operazione) senza eliminare il bisogno di un registro tipo→classe, che la self-registration già fornisce in modo più estendibile.
Utilizzo nel progetto	AIService. <code>request_operation</code> delega alla factory il compito di istanziare l'operazione corretta a partire dal campo <code>type</code> ricevuto dall'API, per ciascuna delle operazioni AI previste (riassunto, traduzione, riscrittura, Distant Writing, sei Cappelli).
Vantaggi	L'aggiunta di una nuova operazione AI richiede solo la creazione di una nuova classe e la sua auto-registrazione, senza modificare AIService né introdurre nuovi condizionali (Open/Closed Principle); il registro rende inoltre immediato esporre a runtime l'elenco delle operazioni disponibili tramite <code>available_types()</code> .

5.1.2 Builder

Problema risolto	La costruzione di un Prompt richiede di comporre più parti opzionali (testo, tipo di operazione, cappello selezionato per l'analisi critica, lingua di destinazione per la traduzione) in modo incrementale, evitando costruttori con molti parametri opzionali.
Soluzione adottata	L'interfaccia PromptBuilder espone metodi fluenti (with_text , with_operation , with_hat , with_language) che restituiscono il builder stesso, e un metodo build() che produce il Prompt finale. StandardPromptBuilder è l'unica implementazione concreta, usata sia come Builder che come Context della Strategy (Sezione 5.3.1): ogni ConcreteStrategy (operazione AI) lo invoca direttamente in build_prompt() . Non è previsto un Director in quanto la sequenza di chiamate al builder varia già correttamente all'interno di ciascun build_prompt() delle diverse operazioni, e un Director fisso duplicherebbe quella logica invece di semplificarla.
Utilizzo nel progetto	Ogni operazione AI usa il builder per assemblare il prompt in base ai soli parametri rilevanti: l'operazione di riassunto usa with_text/with_operation/build , mentre l'analisi secondo i Sei Cappelli aggiunge with_hat .
Vantaggi	Separa la costruzione incrementale del Prompt dalla sua rappresentazione finale, permettendo a ciascuna operazione di comporre solo i parametri per essa rilevanti, senza costruttori sovraccarichi o parametri opzionali non pertinenti.

5.2 Pattern Strutturali

5.2.1 Decorator

Problema risolto	Al servizio di completamento LLM vanno aggiunte, in modo componibile e senza modificarne l'implementazione, funzionalità trasversali come il logging delle richieste/risposte e il caching dei risultati.
Soluzione adottata	LLMServiceDecorator è una classe astratta che realizza LLMService e mantiene un riferimento a un'istanza wrapped: LLMService ; i decorator concreti (CachingLLMAdapter , LoggingLLMAdapter) ne arricchiscono il comportamento prima/dopo aver delegato la chiamata.
Utilizzo nel progetto	La catena effettiva, visibile nel diagramma di sequenza "Richiesta AI (tratto REST→LLM)", è CachingLLMAdapter → LoggingLLMAdapter → OpenAIAdapter (il target concreto), componibile e configurabile all'avvio dell'applicazione senza toccare AIService .
Vantaggi	Le funzionalità trasversali (caching, logging) possono essere aggiunte, rimosse o riordinate componendo la catena in fase di configurazione, senza modificare OpenAIAdapter né AIService , rispettando il Single Responsibility Principle per ciascun anello.

5.2.2 Adapter

Problema risolto	<code>AIService</code> deve rimanere indipendente dal provider LLM concreto; l'interfaccia esposta dal provider esterno (API compatibile con lo standard OpenAI) non coincide con l'interfaccia <code>LLMService</code> usata internamente.
Soluzione adottata	<code>OpenAIAdapter</code> realizza <code>LLMService</code> e incapsula i dettagli di connessione al provider (<code>api_key</code> , <code>model</code> , <code>base_url</code>), traducendo <code>complete(prompt)</code> in una richiesta HTTP verso il servizio LLM e gestendo le eccezioni specifiche (<code>LLMUnavailableError</code> se il provider fallisce, <code>LLMTimeoutError</code> se va in timeout).
Utilizzo nel progetto	Grazie a <code>base_url</code> configurabile, l'adapter permette di puntare indifferentemente ai modelli Gemma, ChatGPT-OSS o Qwen messi a disposizione dall'azienda proponente, senza modifiche al resto del sistema. È al tempo stesso il concrete component su cui si applica la catena di Decorator descritta sopra.
Vantaggi	Isola <code>AIService</code> dai dettagli di uno specifico provider LLM, rendendo possibile sostituire modello o provider (Gemma, ChatGPT-OSS, Qwen) agendo solo sulla configurazione, senza impatti sul resto del dominio applicativo.

5.2.3 Proxy (remoto)

Problema risolto	Il Frontend deve invocare le funzionalità di note-taking e AI esposte dal Backend come se fossero locali, senza spargere nel codice applicativo i dettagli di trasporto (HTTP, base URL, serializzazione).
Soluzione adottata	Le interfacce <code>NoteService</code> e <code>AIService</code> (lato Frontend) definiscono le operazioni disponibili; le rispettive implementazioni Proxy le realizzano incapsulando la comunicazione con, rispettivamente, il file system locale e il Backend REST.
Utilizzo nel progetto	<code>NoteModel</code> dipende solo dall'interfaccia <code>NoteService</code> , <code>AIRequestModel</code> solo dall'interfaccia <code>AIService</code> : nessuna delle due classi di Model conosce se l'implementazione sia locale o remota, facilitando anche il testing tramite implementazioni fittizie. È importante notare che si tratta di un'omonimia intenzionale con la classe concreta <code>AIService</code> del Backend (Sezione 3.4): le due entità vivono a runtime in linguaggi indipendenti e non condividono alcun codice, collegate solo dal contratto REST esposto da <code>AIRouter</code> .
Vantaggi	Il Model rimane completamente ignaro se l'operazione richiesta sia eseguita in locale o tramite rete, semplificando il testing (sostituzione con implementazioni fittizie) e isolando ogni futura evoluzione del trasporto dal resto del Frontend.

5.2.4 Template Method

Problema risolto	L'esportazione di una nota segue sempre gli stessi passi generali (preparazione del contenuto, conversione nel formato di destinazione) ma il dettaglio della conversione dipende dal formato richiesto (PDF, HTML, JSON).
Soluzione adottata	La classe astratta <code>Exporter</code> definisce il metodo <code>template export(content: str): bytes</code> , che fissa la sequenza <code>prepare_content() → convert_format()</code> ; solo <code>convert_format()</code> è il passo variabile, ridefinito da ciascuna sottoclasse (<code>PdfExporter</code> , <code>HtmlExporter</code> , <code>JsonExporter</code>). Un fallimento in fase di conversione solleva <code>ConversionError</code> (sottotipo di <code>ExportError</code>).
Utilizzo nel progetto	<code>ExportRouter.export_note(format, request)</code> risolve tramite <code>DIPProviders.get_exporter(format)</code> l'esportatore concreto e ne invoca <code>export()</code> , senza conoscere il formato specifico: aggiungere un nuovo formato richiede solo una nuova sottoclasse di <code>Exporter</code> .
Vantaggi	Garantisce che ogni formato di esportazione rispetti la stessa sequenza di passi, evitando duplicazione di codice tra gli <code>Exporter</code> concreti; l'aggiunta di un nuovo formato richiede solo l'implementazione di <code>convert_format()</code> , senza toccare la logica comune.

5.3 Pattern Comportamentali

5.3.1 Strategy

Problema risolto	Le operazioni AI condividono la stessa interfaccia di utilizzo (costruzione di un <code>Prompt</code> a partire da un testo) ma differiscono nell'algoritmo di costruzione del prompt stesso; nuove operazioni devono potersi aggiungere senza modificare <code>AIService</code> .
Soluzione adottata	L'interfaccia <code>AIOperation</code> definisce <code>build_prompt(text, params)</code> ; ciascuna operazione concreta (riassunto, traduzione, riscrittura, Distant Writing, le sei varianti dell'analisi critica) la implementa con la propria logica, usando <code>StandardPromptBuilder</code> come contesto di costruzione.
Utilizzo nel progetto	<code>AIService</code> non conosce le operazioni concrete: riceve un'istanza dalla Simple Factory e la invoca in modo polimorfo, come illustrato nel diagramma di sequenza "Richiesta AI (tratto REST→LLM)".
Vantaggi	Permette di aggiungere nuove operazioni AI, incluse eventuali varianti dei Sei Cappelli, come nuove classi che implementano <code>AIOperation</code> , senza modificare <code>AIService</code> (Open/Closed Principle).

5.3.2 Command

Problema risolto	Le modifiche al testo della nota (formattazione, tabelle, elenchi, link, inserimento) devono poter essere annullate e ripetute in modo uniforme (R43-F-O, R44-F-O), senza che <code>NoteModel</code> conosca i dettagli di ciascuna operazione di editing.
Soluzione adottata	Un'interfaccia <code>EditCommand</code> definisce <code>execute()</code> / <code>undo()</code> ; ogni tipo di modifica la implementa incapsulando lo stato necessario per l'annullamento. Una <code>CommandHistory</code> mantiene due pile (undo/redo).
Utilizzo nel progetto	Il metodo <code>executeCommand(c)</code> di <code>NoteModel</code> delega a <code>CommandHistory</code> , che esegue e archivia il comando; l'accettazione di una proposta AI riusa lo stesso meccanismo, inserendo il testo generato tramite un <code>InsertTextCommand</code> – lo stesso comando usato per l'inserimento manuale di testo (diagramma di sequenza “Accetta proposta AI”).
Vantaggi	Uniforma undo/redo per qualsiasi tipo di modifica testuale, manuale o generata dall'AI, attraverso un'unica interfaccia, evitando di duplicare la logica di annullamento tra editing manuale e accettazione delle proposte AI.

5.3.3 Observer

Problema risolto	Model e View devono notificare i propri cambiamenti a più osservatori (View, Controller) senza conoscerne il tipo concreto, per mantenere il disaccoppiamento richiesto dal pattern MVC (Sezione 3.3).
Soluzione adottata	Un'interfaccia <code>Subject</code> definisce <code>attach/detach/notify</code> ; un'interfaccia <code>Observer</code> definisce <code>update()</code> . Il Model realizza <code>Subject</code> ; la View realizza sia <code>Subject</code> sia <code>Observer</code> ; il Controller realizza <code>Observer</code> .
Utilizzo nel progetto	Ogni modifica al contenuto della nota o allo stato di una richiesta AI propaga automaticamente l'aggiornamento a View e Controller, garantendo la visualizzazione continua dello stato richiesta da R72-F-O.
Vantaggi	Disaccoppia il Model dai suoi osservatori: nuovi componenti potrebbero sottoscrivere agli aggiornamenti di stato senza richiedere modifiche a <code>NoteModel</code> o <code>AIRequestModel</code> .

5.3.4 Dependency Injection

Problema risolto	I router FastAPI devono ottenere le proprie dipendenze (AIService , esportatori per formato) senza costruirle direttamente, per restare testabili e disaccoppiati dalla configurazione concreta (provider LLM, formato di esportazione).
Soluzione adottata	DIProviders espone funzioni fabbrica (get_ai_service , get_exporter) iniettate nei router dal meccanismo di Dependency Injection nativo di FastAPI (Depends).
Utilizzo nel progetto	AIRouter ed ExportRouter dipendono da DIProviders per ottenere le proprie collaborazioni a runtime, invece di istanziarle staticamente.
Vantaggi	Rende i router testabili in isolamento iniettando implementazioni fittizie di AIService o degli esportatori, e permette di sostituire la configurazione concreta (es. provider LLM) senza modificare il codice dei router.

6 Flusso dei Dati (Data Flow)

Questa sezione illustra il ciclo di vita delle informazioni all'interno del sistema, partendo dall'interazione dell'utente fino all'ottenimento del risultato generato dall'AI o alla persistenza della nota.

Le frecce continue con punta piena rappresentano chiamate sincrone, le frecce tratteggiate le risposte (reply), le frecce continue con punta stilizzata rappresentano chiamate asincrone, infine, quelle contrassegnate «create» istanziano un nuovo oggetto comando.

6.1 Elaborazione di un'operazione AI

Il flusso, dalla richiesta dell'utente alla visualizzazione della proposta, attraversa tre diagrammi di sequenza in successione, ciascuno con una responsabilità netta: il primo e il terzo restano nel Frontend (rispettivamente l'avvio della richiesta e la ricezione della risposta), il secondo copre l'intero tratto di rete verso il Backend e l'LLM esterno.

1a – Richiesta operazione AI (tratto Frontend). L'utente seleziona il testo e richiede un'operazione (es. "Riassumi"); la View notifica il Controller, che invoca il metodo `requestAIOperation()` di `AIRequestModel`. Il Model passa immediatamente allo stato `Processing` e notifica gli observer, mostrando all'utente uno spinner e l'opzione di interruzione (R72-F-O); la richiesta viene poi inviata in modo asincrono ad `AIServiceProxy`.

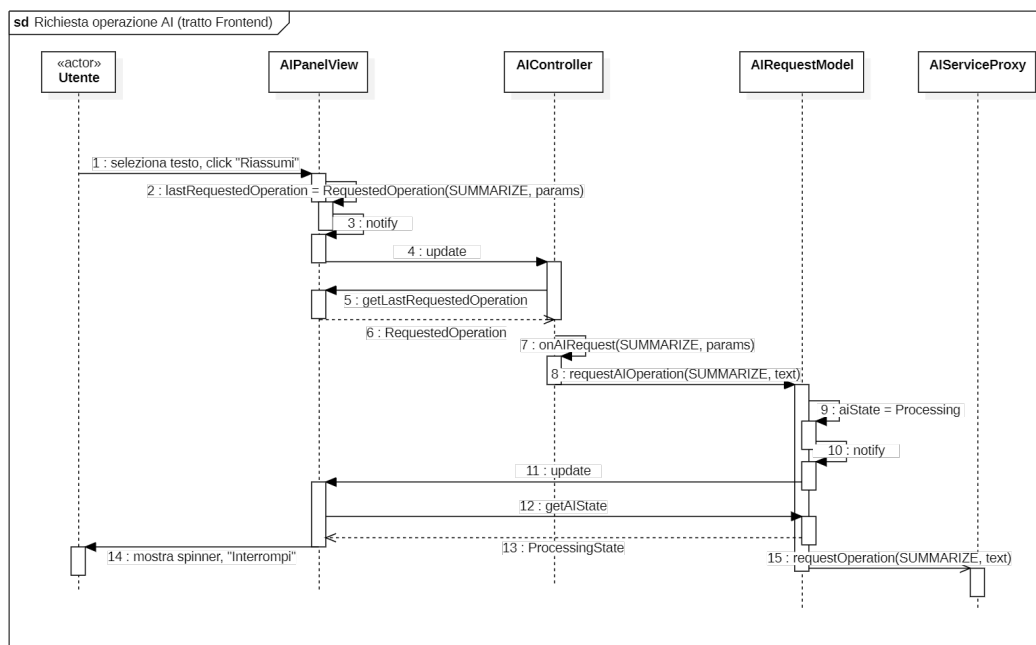


Figura 9: Richiesta operazione AI (tratto Frontend).

1b – Richiesta AI (tratto REST→LLM). `AIRouter` riceve la richiesta e la inoltra ad `AIService`, che risolve la strategia corretta tramite la Simple Factory e costruisce un `Prompt` tramite `PromptBuilder`. La chiamata attraversa poi la catena di decorator (cache, logging) fino all'`OpenAIAdapter`, che contatta il servizio LLM esterno; la risposta risale la stessa catena fino a `AIService`, che costruisce la `Proposal` e la restituisce come `AIOperationResponse`.

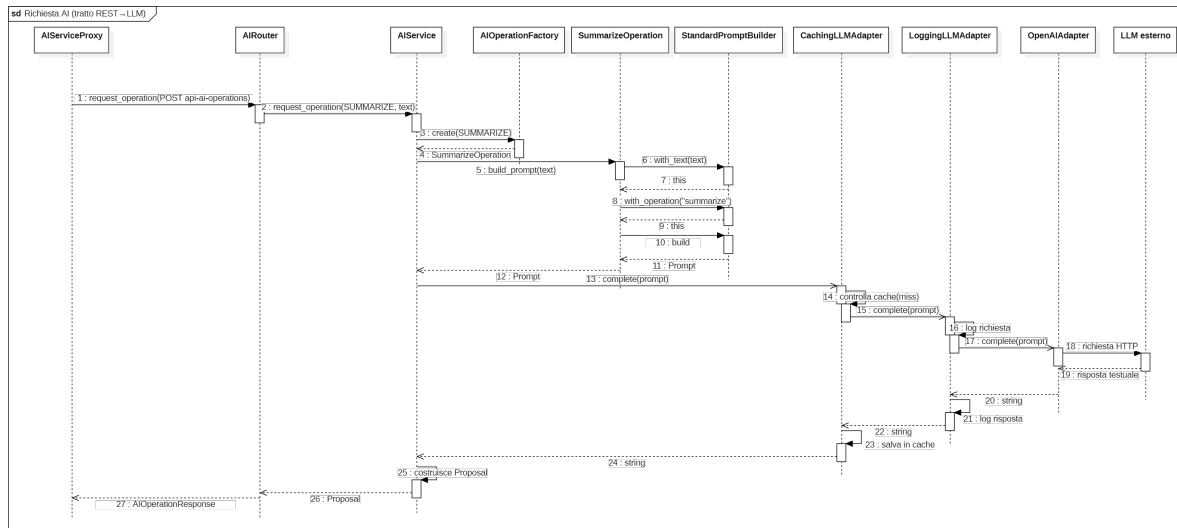


Figura 10: Richiesta AI (tratto REST→LLM): attraversa in un unico flusso i pattern Strategy, Builder, Simple Factory, Decorator e Adapter.

1c – Ricezione proposta AI (tratto Frontend). La risposta HTTP, una volta risolta la Promise di `AIServiceProxy.requestOperation()`, riporta il controllo al Model: `AIRequestModel` transita allo stato `ProposalReadyState(proposal)` e notifica gli observer; `AIPanelView` interroga lo stato aggiornato e mostra all'utente la proposta con le azioni disponibili (Accetta/Rifiuta/Rigenera).

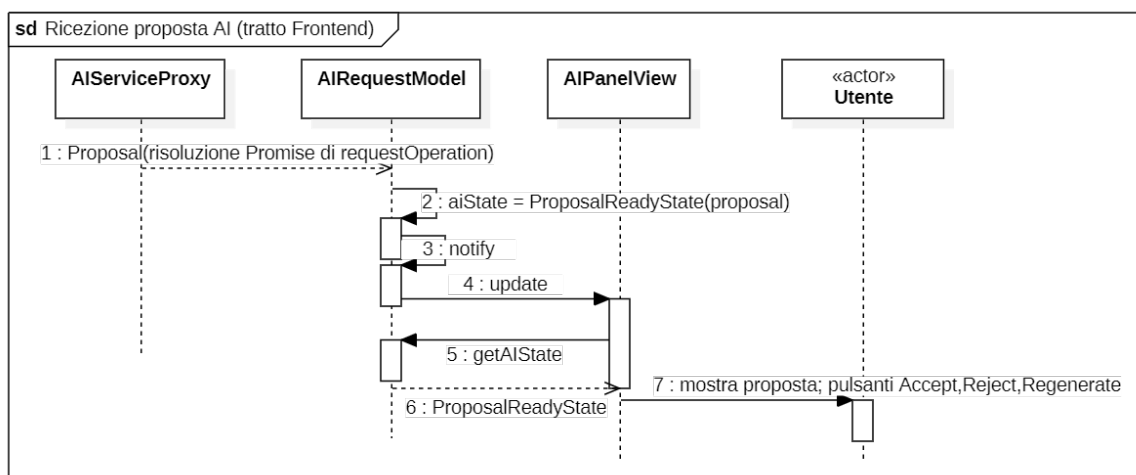


Figura 11: Ricezione proposta AI (tratto Frontend).

6.2 Accettazione della proposta ed elenco delle operazioni

Accetta proposta AI (riuso Command). Se l'utente accetta la proposta, `AIController` recupera il testo da `AIRequestModel` (stato `ProposalReadyState`) e crea un `InsertTextCommand` (lo stesso comando usato per l'inserimento manuale di testo) eseguito tramite

`NoteModel.executeCommand()`. Il Model torna quindi allo stato `Idle` e il pannello proposta si chiude.

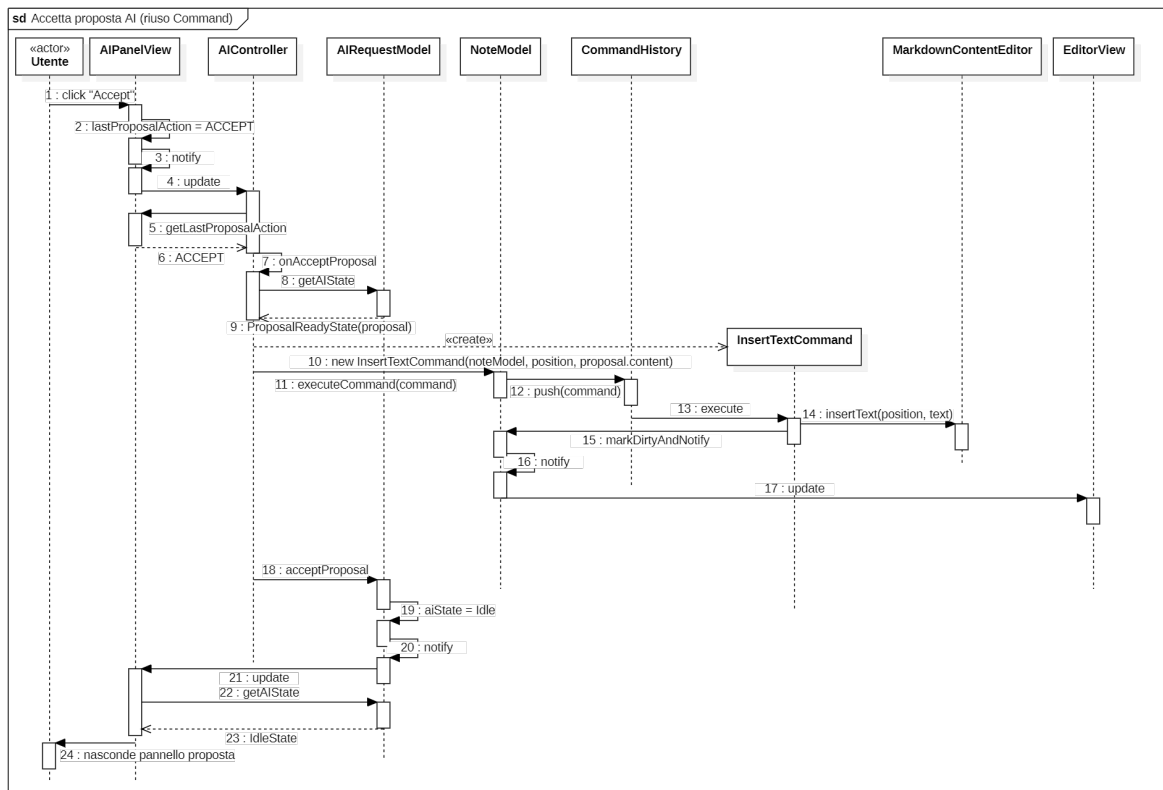


Figura 12: Accetta proposta AI (riuso del pattern Command).

Elenco operazioni disponibili (Simple Factory self-registration). L'elenco delle operazioni mostrate nel selettore del Frontend non è hard-coded: `AIPanelView` lo richiede a runtime, risalendo la catena fino ad `AIOperationFactory.available_types()`, che legge il proprio registro di auto-registrazione. Aggiungere una nuova operazione AI non richiede quindi alcuna modifica a questo flusso.

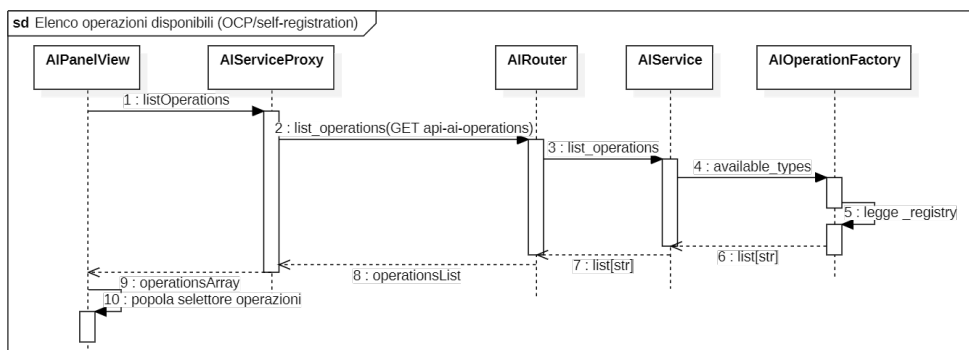


Figura 13: Elenco operazioni disponibili (Simple Factory self-registration).

6.3 Editing del testo e Undo/Redo

I tre diagrammi seguenti mostrano il riuso dello stesso meccanismo Command/CommandHistory per operazioni di editing diverse (formattazione, tabelle) e per il relativo undo/redo.

Formattazione con Undo. La View registra la richiesta dell'utente (es. "Grassetto"), si auto-notifica, il Controller la preleva e crea il **FormatTextCommand** corrispondente, eseguito e archiviato da **CommandHistory**. L'Undo riesegue in ordine inverso l'ultimo comando in pila, ripristinando il contenuto precedente tramite lo stesso oggetto comando.

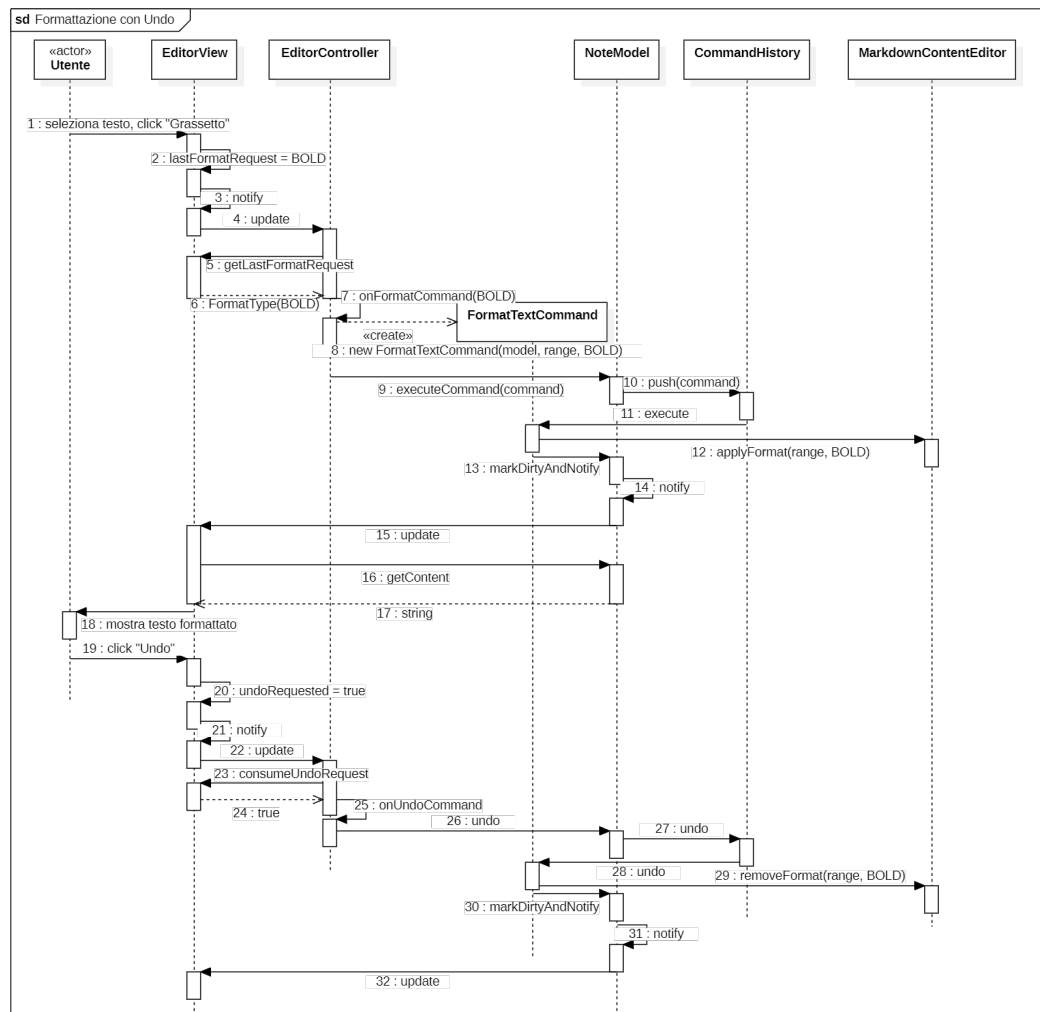


Figura 14: Formattazione di un testo e relativo Undo.

Redo. Continuazione dello scenario precedente: il Redo rieseguo l'ultimo comando spostato dalla pila di redo a quella di undo.

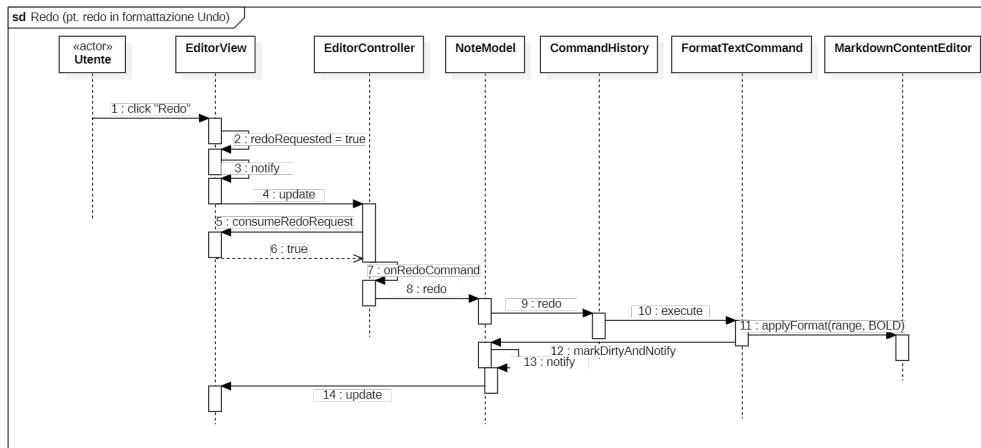


Figura 15: Redo (continuazione di “Formattazione con Undo”).

Inserimento tabella con validazione. Il frame `alt` distingue due rami alternativi: se le dimensioni richieste (righe/colonne) non sono valide, `MarkdownContentEditor` solleva `InvalidTableDimensionError`, propagato fino alla View che mostra l'errore senza modificare il documento; se le dimensioni sono valide, la tabella viene inserita e la nota aggiornata normalmente.

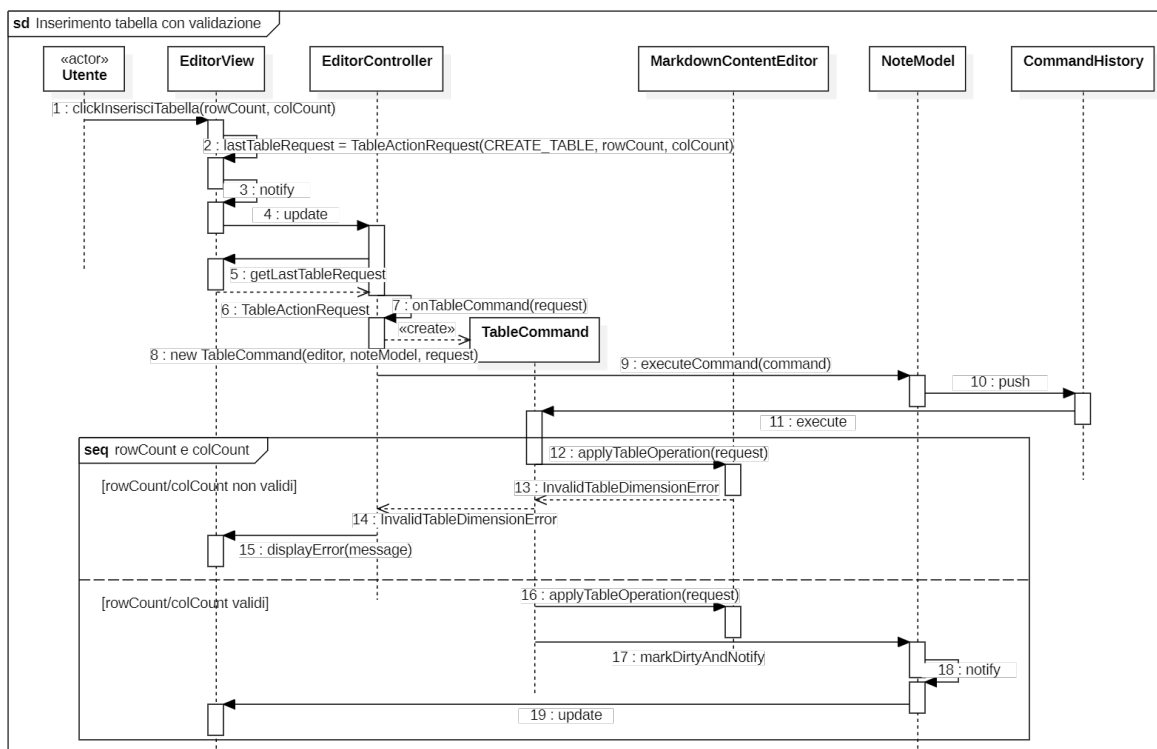


Figura 16: Inserimento di una tabella, con validazione delle dimensioni.

6.4 Persistenza locale della nota

Salvataggio nota locale. Il frame `alt` distingue, in base alla presenza di un identificativo di nota, due casi: se la nota non ha ancora un file associato (`noteId` nullo), `NoteServiceProxy` apre il selettore di salvataggio del browser (`showSaveFilePicker`) prima di scrivere; se la nota è già associata a un file, scrive direttamente (R75-F-O).

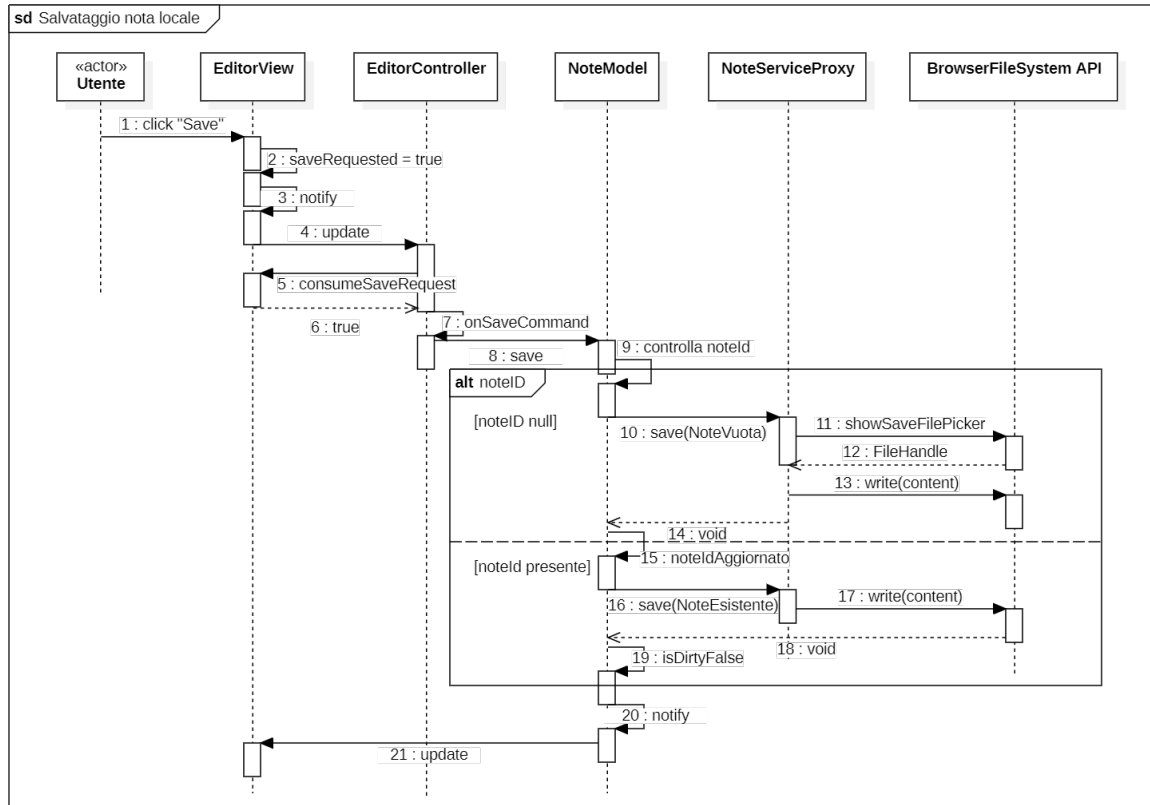


Figura 17: Salvataggio di una nota in locale.

Salvataggio con errore. In caso di errore di I/O durante la scrittura, l'eccezione `NoteIOError` risale da `BrowserFileSystemAPI` fino alla View, che notifica l'utente mantenendo la nota invariata.

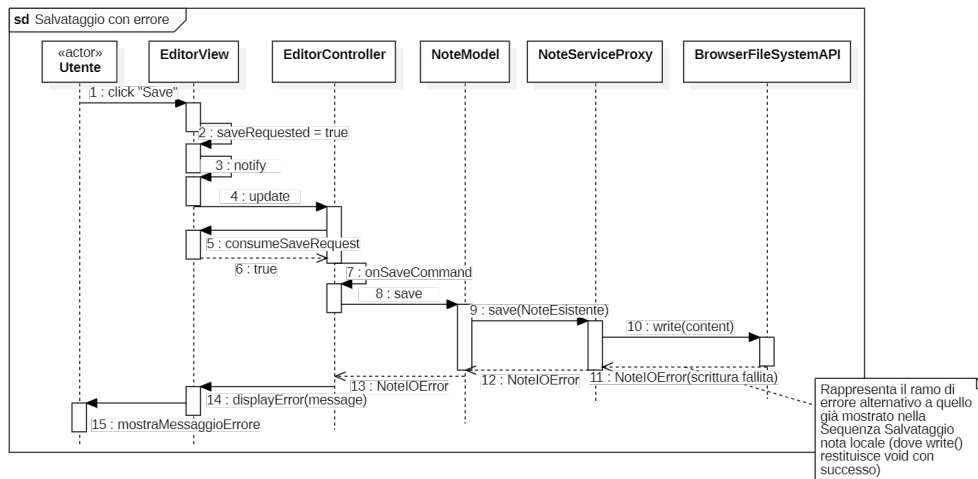


Figura 18: Salvataggio con errore di I/O.

Importazione/Apertura nota. L'apertura di un file esistente (R76-F-O) passa dal selettore del browser (`showOpenFilePicker`), la lettura del contenuto, la ricostruzione dell'oggetto `Note` e l'aggiornamento di `MarkdownContentEditor`; la `CommandHistory` viene azzerata per non permettere un Undo o Redo che attraversino note diverse.

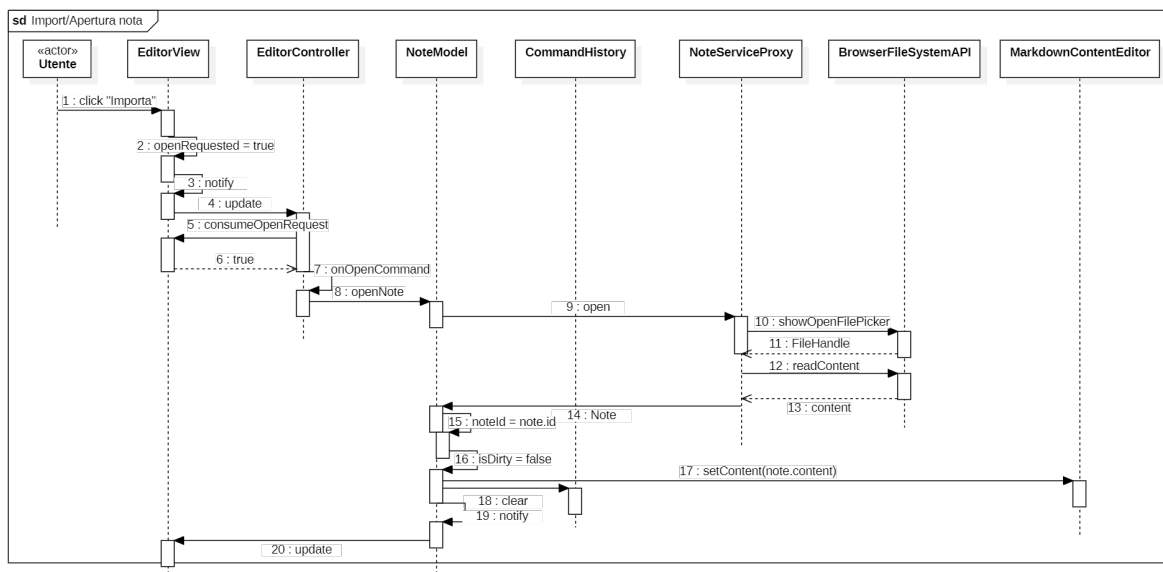


Figura 19: Importazione/Apertura di una nota esistente.

6.5 Esportazione

Export PDF (Template Method). `ExportRouter` delega a `PdfExporter.export()`, che segue lo scheletro fisso definito da `Exporter`: prepara il contenuto (`prepare_content`) e ne converte solo il formato finale (`convert_format`), l'unico passo ridefinito dalla sottoclasse.

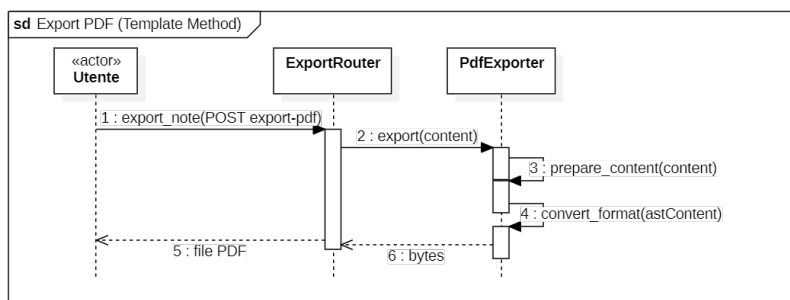


Figura 20: Esportazione di una nota in PDF (Template Method).

7 Tracciamento e Stato dei Requisiti

Al fine di garantire che l'architettura progettata e le tecnologie scelte soddisfino le funzionalità concordate, si fornisce di seguito la tracciabilità tra le componenti architetturali descritte in questo documento e i requisiti individuati in [Analisi dei Requisiti](#) (Sezione 4 – Requisiti), a cui si rimanda per la descrizione testuale completa di ciascun requisito.

7.1 Tabella Tracciamento Componenti-Requisiti

Tabella 1: Tracciamento Componenti → Requisiti

Componente	Responsabilità	Requisiti coperti
MarkdownContent-Editor, EditorView/EditorController	Scrittura libera, copia/taglia/incolla, formattazione (grassetto, corsivo, sottolineato, barrato, citazione, intestazione), link, elenchi e tabelle	R1–R42
EditCommand (famiglia), CommandHistory	Annullamento e ripristino delle modifiche	R43-F-O, R44-F-O
EditorView (modalità di visualizzazione)	Visualizzazione esclusiva editor/render o affiancata (Split View)	R45–R47
AIRouter, AIService, AIOperationFactory, operazioni Strategy	Riassunto, traduzione, riscrittura, Distant Writing e analisi Sei Cappelli tramite LLM	R48, R50, R52, R53, R55, R57, R59, R61, R63, R65, R67
ErrorHandlers, gerarchia AIDomainError	Notifica degli errori di comunicazione con i servizi AI	R49, R51, R54, R56, R58, R60, R62, R64, R66, R68
AIRequestModel, AIPanelView, AIController	Accettazione/rifiuto/rigenerazione della proposta, interruzione manuale, stato di avanzamento continuo	R69–R74
NoteModel, NoteService/NoteServiceProxy	Salvataggio e caricamento della nota in formato Markdown dal file system locale	R75-F-O, R76-F-O
ExportRouter, esportatori per formato (Template Method)	Esportazione della nota in PDF, HTML e JSON	R77-F-O
Suite di test automatici, pipeline CI	Copertura di test, documentazione tecnica e di qualifica	R1-Q-O – R9-Q-D
Architettura Client-Server, adapter LLM	Esecuzione via browser, standard API OpenAI, modelli Gemma/ChatGPT-OSS/Qwen	R1-V-O – R7-V-O
Continua nella pagina successiva...		

Tabella 1 – Continua dalla pagina precedente

Componente	Responsabilità	Requisiti coperti
Rendering reattivo Vue.js, stato AIRequestState	Aggiornamento anteprima entro 1s, indicatore di caricamento continuo	R1-P-O, R2-P-O

I requisiti opzionali R4-V-Op e R78–R86 (persistenza remota, gestione utenti) non compaiono in questa tabella poiché il gruppo ha deciso di non implementarli in questa fase (si veda Sezione 7.2 per lo stato dettagliato e la motivazione): non esiste quindi alcun componente architetturale a cui tracciarli.

7.2 Stato di Soddisfacimento dei Requisiti

La tabella seguente riassume, per macro-gruppo, lo stato di implementazione dei requisiti funzionali, di qualità, di vincolo e prestazionali rispetto all'architettura descritta in questo documento. I gruppi ricalcano la struttura di [Analisi dei Requisiti](#), Sezione 4; il numero di requisiti obbligatori (90), desiderabili (2) e opzionali (12) è riepilogato nello stesso documento, Sezione 6 – Riepilogo.

I requisiti opzionali relativi alla persistenza remota e alla gestione utenti (R4-V-Op, R7-Q-Op, R8-Q-Op, R78–R86) sono stati valutati e volutamente esclusi dallo scope della release corrente: il gruppo ha scelto di concentrare le risorse disponibili sul consolidamento e sulla qualità dei requisiti obbligatori, rimandando un'eventuale implementazione a fasi successive del progetto qualora il tempo residuo lo consenta. Sono pertanto riportati con stato *Non implementato*.

Tabella 2: Stato dei Requisiti

Codici	Descrizione	Tipo	Stato
R1–R4	Scrittura, copia, taglia, incolla di testo nell'editor.	Obbligatorio	Soddisfatto
R5–R28	Attivazione / applicazione / disattivazione / rimozione delle formattazioni grassetto, corsivo, sottolineato, barrato, citazione e intestazione.	Obbligatorio	Soddisfatto
R29–R31	Inserimento, modifica e rimozione di link ipertestuali.	Obbligatorio	Soddisfatto
R32–R35	Inserimento, modifica, disattivazione e rimozione di elenchi puntati e numerati.	Obbligatorio	Soddisfatto
R36–R42	Inserimento e modifica di tabelle Markdown, con validazione dei parametri.	Obbligatorio	Soddisfatto
R43, R44	Annullamento (Undo) e ripristino (Redo) dell'ultima modifica.	Obbligatorio	Soddisfatto
Continua nella pagina successiva...			

Tabella 2 – Continua dalla pagina precedente

Codici	Descrizione	Tipo	Stato
R45–R47	Visualizzazione esclusiva editor, render o affiancata (Split View).	Obbligatorio	Soddisfatto
R48, R49	Generazione riassunto tramite LLM e notifica di errore.	Obbligatorio	Soddisfatto
R50, R51	Traduzione (Inglese, Francese, Spagnolo) tramite LLM e notifica di errore.	Obbligatorio	Soddisfatto
R52	Traduzione in Tedesco come estensione facoltativa.	Desiderabile	Soddisfatto
R53, R54	Riscrittura del testo tramite LLM e notifica di errore.	Obbligatorio	Soddisfatto
R55, R56	Distant Writing da prompt utente e notifica di errore.	Obbligatorio	Soddisfatto
R57–R68	Analisi critica secondo le sei prospettive dei Cappelli di De Bono e relative notifiche di errore.	Obbligatorio	Soddisfatto
R69–R71	Accettazione, rifiuto e rigenerazione della proposta AI.	Obbligatorio	Soddisfatto
R72, R73	Visualizzazione continua dello stato di avanzamento e interruzione manuale della richiesta AI.	Obbligatorio	Soddisfatto
R74	Visualizzazione della proposta AI in un modale dedicato.	Obbligatorio	Soddisfatto
R75, R76	Salvataggio e caricamento della nota in formato Markdown su file system locale.	Obbligatorio	Soddisfatto
R77	Esportazione della nota in PDF, HTML e JSON.	Obbligatorio	Soddisfatto
R78–R86	Creazione/eliminazione nota, registrazione/autenticazione utente, gestione note remote.	Opzionale	Non implementato
R1-Q-O – R6-Q-O	Copertura di test, documentazione tecnica, versionamento Git, test di integrazione con i servizi LLM.	Obbligatorio	Soddisfatto
R7-Q-Op, R8-Q-Op	Test di integrazione e autenticazione del modulo opzionale di persistenza remota.	Opzionale	Non implementato
R9-Q-D	Adozione di una licenza open source permissiva.	Desiderabile	Soddisfatto
R1-V-O – R3-V-O	Esecuzione via browser standard e interazione con i modelli LLM tramite API compatibile OpenAI.	Obbligatorio	Soddisfatto
Continua nella pagina successiva...			

Tabella 2 – Continua dalla pagina precedente

Codici	Descrizione	Tipo	Stato
R4-V-Op	Persistenza remota tramite un database server-side.	Opzionale	Non implementato
R5-V-O – R7-V-O	Compatibilità browser, singolo modello LLM per richiesta, repository pubblico.	Obbligatorio	Soddisfatto
R1-P-O	Aggiornamento dell’anteprima Mark-down entro 1 secondo dalla modifica.	Obbligatorio	Soddisfatto
R2-P-O	Visualizzazione continua e reattiva dello stato di caricamento durante l’elaborazione AI.	Obbligatorio	Soddisfatto